

Event-Related MCP Agents for Asynchronous Enterprise Systems

Siva Rama Krishna Varma Bayyavarapu
Independent Researcher
Indiana, United States
siva.bayyavarapu@ieee.org

Vishwanath Hiremath
Principal System/Software Engineer
California, United States
hiremath1@ieee.org

Vijayakumar Venganti
Independent Researcher
San Jose, CA
vijayakumar.venganti@ieee.org

Abstract—Modern Model Context Protocol (MCP) agents are primarily designed around synchronous tool invocation models, assuming immediate response availability and short-lived execution cycles. However, enterprise systems operate in highly asynchronous, event-driven environments where operations span multiple services, external platforms, and delayed completion events. This architectural mismatch leads to lost context, duplicated execution, reduced SLA compliance, and fragile coupling between agents and enterprise infrastructure. This paper introduces an Event-Related MCP Agent architecture that enables agents to operate natively within asynchronous enterprise ecosystems. The proposed design externalizes intent publication, maintains correlation-aware execution context, and supports resumable reasoning triggered by distributed event streams. The framework incorporates deferred context reconciliation, SLA-aware recovery, and fault-isolated orchestration mechanisms. Discrete-event simulation (500 workflows, 4 services, 20% failure) demonstrates 90% reduction in blocking time, 13 percentage points higher SLA compliance, and higher modeled operation throughput under bounded concurrency assumptions compared to traditional synchronous tool-calling agents. The proposed architecture provides a scalable, enterprise-oriented foundation for adapting intelligent agents to asynchronous distributed systems.

Index Terms—MCP agents, event-driven systems, asynchronous orchestration, correlation identifiers, SLA compliance, enterprise systems, resumable reasoning, distributed workflow execution.

I. INTRODUCTION

Agent-based systems leveraging the Model Context Protocol (MCP) [1] have gained adoption for tool orchestration and contextual task execution. Most current implementations rely on synchronous request-response patterns, assuming immediate tool availability and deterministic completion behavior. Enterprise environments, however, are characterized by event-driven communication, multi-stage processing, external dependency latency, distributed state ownership, and eventual consistency models [2], [3]. The disconnect between synchronous agent models and asynchronous enterprise workflows introduces operational fragility—lost context, duplicated execution, reduced SLA compliance, and brittle coupling [4].

In industry environments, this problem appears in common operational workflows such as contract approvals, fulfillment pipelines, compliance checks, incident remediation, customer provisioning, and external system integrations. These workflows rarely complete within a single synchronous request. Instead, they depend on queues, event buses, third-party APIs,

retries, delayed callbacks, and human-in-the-loop actions. When an agent is designed only around immediate tool responses, it cannot reliably preserve execution state across these delayed boundaries. This creates practical risks: duplicated actions, missed completion events, manual reconciliation, SLA violations, and weak observability across distributed services. The proposed event-related model addresses this industry gap by treating agent intent as a durable event, storing correlation-aware context, and resuming reasoning only after matching completion or state-change events are received.

This work addresses the following research question: *How can MCP agents be architected to operate reliably within asynchronous, event-driven enterprise systems while preserving loose coupling and SLA compliance?*

For clarity, we use the following acronyms throughout the paper: MCP refers to the Model Context Protocol, CID refers to a correlation identifier, SLA refers to a service-level agreement, and EDA refers to event-driven architecture.

Our contributions are:

(1) An Event-Related MCP Agent architecture with five core components—Intent Publisher, Correlation Context Store, Event Subscription Engine, Reconciliation Engine, and Resumable Reasoning Layer—that enables native operation in asynchronous enterprise ecosystems.

(2) A formal correlation model $C = (I, CID, S, T, M)$ for deterministic event-to-intent matching and resumable execution, with an SLA-aware recovery model M_{sla} for latency thresholds, retry policies, and fallback strategies.

(3) Discrete-event simulation (500 workflows, 4 services, 20% failure) demonstrating 90% reduction in blocking time, 13 percentage points higher SLA compliance, and higher modeled throughput under bounded concurrency versus traditional synchronous MCP agents.

(4) Design patterns and integration guidance for event buses, microservices, and cloud-native infrastructure.

The remainder of this paper is organized as follows. Section II surveys related work in event-driven systems and agent frameworks. Section III presents background and motivation. Section IV formulates the problem. Section V describes the proposed architecture. Section VI details the correlation and SLA-aware recovery models. Section VII presents experimental evaluation. Section VIII discusses implications and limitations. Section IX concludes.

II. RELATED WORK

A. Agent Tool Invocation Frameworks

The Model Context Protocol (MCP) [1] standardizes how AI applications provide tools and context to language models. OpenAI and Anthropic support MCP for tool invocation via hosted, HTTP, and stdio transports. Current MCP implementations assume synchronous tool calls: the agent invokes a tool and blocks until a response is returned. This design fails when tools wrap long-running enterprise operations—order fulfillment, multi-step approvals, or external API integrations—that complete asynchronously [5].

Prior work has also explored safe and policy-compliant multi-agent orchestration in enterprise AI environments, focusing on policy feasibility, bounded risk, and coordinated execution; however, event-correlated resumption across asynchronous enterprise workflows remains a separate systems challenge.

B. Event-Driven and Microservices Orchestration

Event-driven architectures [2], [6] use message buses and publish-subscribe patterns to decouple producers and consumers. Correlation IDs and distributed tracing [7] enable end-to-end tracing across distributed services. OpenTelemetry [8] and W3C TraceContext [9] define standard propagation for linking spans across asynchronous flows. Orchestration systems such as Temporal [10], Conductor [11], and Argo Workflows [12] support long-running workflows but delegate scheduling to underlying runtimes and do not integrate agent reasoning with event correlation.

Related work on transaction-intensive cloud AI workflows highlights the need to automate streaming-to-predictive processing pipelines in high-volume digital ecosystems; our work addresses the complementary problem of preserving agent intent and reasoning state across delayed event boundaries [17].

C. SLA-Aware Resource Management

SLA-aware scheduling [13]–[15] addresses deadline compliance and resource allocation. HAVEN [16] provides holistic load balancing and auto scaling in the cloud. These systems focus on capacity and task ordering rather than agent-intent correlation and resumable reasoning. To our knowledge, no prior work integrates MCP agents with event-correlated reasoning, deferred reconciliation, and SLA-aware recovery in a unified architecture.

D. Comparison with Closest Approaches

Table I summarizes how our Event-Correlated MCP Agent compares to the closest related systems. MCP [1] and orchestration frameworks [10]–[12] lack event correlation and resumable reasoning; HAVEN [16] optimizes capacity but not agent-intent correlation. Our architecture uniquely combines intent publication, CID-based reconciliation, and SLA-aware recovery.

Although Temporal, Conductor, and Argo support asynchronous workflow orchestration, they are primarily workflow

execution engines rather than MCP-native agent reasoning systems. Our goal is not to replace these frameworks, but to define an agent-facing architecture that connects intent publication, CID-based event reconciliation, and resumable reasoning. In production, the proposed model can be implemented on top of such orchestration engines. A full empirical comparison with Temporal-, Conductor-, or Argo-backed implementations is left as future work.

TABLE I
COMPARISON WITH RELATED SYSTEMS

System	Event	Resum.	Corr.	SLA
MCP [1]	No	No	No	No
Temporal [10]	Yes	Yes (wf)	No	No
Conductor [11]	Yes	Yes (wf)	No	No
HAVEN [16]	No	No	No	Yes
Event-Corr. MCP	Yes	Yes	Yes (CID)	Yes

III. BACKGROUND AND MOTIVATION

A. Limitations of Synchronous MCP Agents

Traditional MCP agents invoke tools directly, block execution awaiting response, assume immediate system state availability, and maintain transient context in-memory. These assumptions fail in scenarios involving long-running workflows, external system dependencies, multi-service orchestration, and delayed completion signals. When a tool wraps an asynchronous operation, the agent either blocks indefinitely or loses context when the operation completes later.

B. Characteristics of Enterprise Systems

Enterprise distributed systems commonly employ event buses, microservices, asynchronous messaging, retry queues, observability-driven recovery, and SLA-governed APIs [2], [3]. Agents must conform to these properties to achieve production reliability. The proposed architecture bridges this gap.

IV. PROBLEM STATEMENT

Given: (a) a distributed enterprise system composed of services $S = \{s_1, s_2, \dots, s_n\}$; (b) an event bus E emitting asynchronous signals; and (c) an MCP agent A invoking tool set T , we observe:

- **Traditional execution model:** $A \rightarrow t_i \rightarrow \text{response} \rightarrow \text{next action (blocking)}$.
- **Enterprise execution model:** $A \rightarrow \text{publish intent} \rightarrow \text{asynchronous processing} \rightarrow \text{event emission} \rightarrow \text{reconciliation} \rightarrow \text{continuation (non-blocking)}$.

The problem arises when the agent cannot: (1) persist structured execution context; (2) correlate delayed events with originating intent; (3) resume reasoning deterministically; or (4) isolate failure domains. We define the need for an event-correlated execution model.

V. PROPOSED ARCHITECTURE

A. Architectural Overview

The Event-Related MCP Agent introduces five core components, shown in Fig. 1:

- 1) **Intent Publisher** — Serializes agent intent and publishes to the event bus with a correlation identifier.
- 2) **Correlation Context Store** — Persists execution context (I, CID, S, T, M) for later reconciliation.
- 3) **Event Subscription Engine** — Subscribes to completion and state-change events from the event bus.
- 4) **Reconciliation Engine** — Matches incoming events to stored contexts by CID and triggers resumption.
- 5) **Resumable Reasoning Layer** — Restores agent state and continues reasoning when reconciliation succeeds.

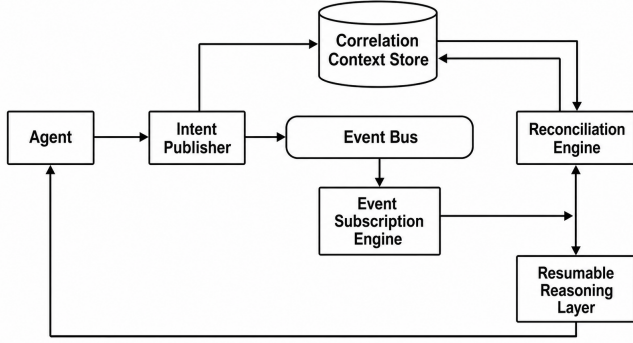


Fig. 1. Event-Related MCP Agent architecture. The agent publishes intent via the Intent Publisher; the Correlation Context Store persists state; the Event Subscription Engine receives completion events; the Reconciliation Engine matches events to contexts; the Resumable Reasoning Layer continues execution.

B. Execution Flow

Fig. 2 illustrates the seven-step execution flow:

- 1) Agent generates structured intent.
- 2) Intent is published to the event bus.
- 3) Correlation ID is stored persistently in the Correlation Context Store.
- 4) Downstream services process asynchronously.
- 5) Completion or state-change events are emitted to the event bus.
- 6) Event Subscription Engine receives events; Reconciliation Engine matches $e.CID$ to stored CID.
- 7) Resumable Reasoning Layer restores context and resumes agent reasoning.

This removes blocking dependency on synchronous tool invocation. The agent yields control after publishing intent and resumes only when a matching event arrives.

VI. CORRELATION AND SLA-AWARE RECOVERY MODELS

A. Correlation Model

Let each execution instance be represented as:

$$C = (I, CID, S, T, M) \quad (1)$$

where I = intent payload, CID = correlation identifier, S = service interaction set, T = temporal state, and M = metadata (SLA, retry policy, identity scope). The agent resumes execution only when Event(e) such that $e.CID = CID$. This enables deterministic continuation.

Example: An agent orders parts from supplier s_1 ($CID = \text{"ord-7f3a"}$). The intent $I = \{\text{action: "order", sku: "P-42", qty: 10}\}$. When s_1 emits completion event e with $e.CID = \text{"ord-7f3a"}$, the Reconciliation Engine matches and triggers resumption with the fulfillment result.

B. Correctness and Scalability

The correctness of the correlation model follows from the invariant that each resumable execution context is persisted with a unique CID before intent publication. An incoming event can resume execution only when its event CID matches an existing stored context. Therefore, unrelated or delayed events cannot trigger an incorrect continuation. Under at-least-once delivery, duplicate events are handled idempotently by recording the last completed step for each CID, ensuring that repeated completion events do not advance or corrupt agent state multiple times.

From a scalability perspective, reconciliation can be implemented as an indexed lookup over CID in the Correlation Context Store. With a key-value or document-store implementation, event-to-context matching is expected to be $O(1)$ per incoming event, while storage grows linearly with the number of active or retained execution contexts. Retention policies, TTL-based cleanup, and archival of completed contexts can be used to bound long-term storage growth.

C. SLA-Aware Recovery Model

The system integrates SLA metadata:

$$M_{sla} = (\text{latency_threshold}, \text{retry_limit}, \text{fallback_priority}) \quad (2)$$

When latency exceeds the threshold: (1) retry policy invoked; (2) alternative tool selected if configured; (3) degraded mode activated. This enables dynamic resilience without synchronous blocking. Under at-least-once delivery, completion events may be duplicated; the Reconciliation Engine treats reconciliation as idempotent (e.g., using *last_completed_step* per CID) so duplicate events are applied once and do not corrupt state. Fail-secure behavior: on Correlation Context Store unavailability, the system rejects new intent publication and logs alerts rather than proceeding with lost context.

VII. EXPERIMENTAL EVALUATION

A. Setup

We implemented a discrete-event simulation modeling a multi-stage workflow across 4 services. Each service incurs latency drawn uniformly from 100 ms–900 ms. We inject failures at 20% per service call. The SLA threshold is 3000 ms.

We define the concurrency parameter N as the number of concurrent workflows. For the event-correlated agent we

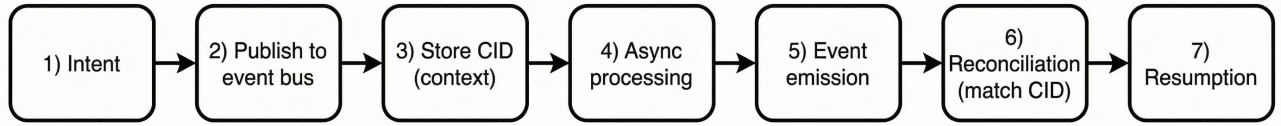


Fig. 2. Execution flow: intent publication → persistent storage → asynchronous processing → event emission → reconciliation → resumption.

use $N = 10$. The backend is assumed non-saturated (service capacity suffices for N concurrent requests).

The synchronous baseline reflects the default blocking tool-calling model used in current MCP-style implementations. While optimized asynchronous baselines may reduce the observed gap, they do not eliminate the fundamental limitation of lacking persistent correlation-aware resumption.

Each configuration was repeated 30 times with different random seeds. We report mean values with 95% confidence intervals. Statistical significance between synchronous and event-correlated models was evaluated using the Mann–Whitney U test.

B. Results

TABLE II
SIMULATION RESULTS: SYNCHRONOUS VS. EVENT-CORRELATED MCP AGENTS (30 RUNS, 95% CI)

Metric	Sync.	Event-Corr.
Avg Blocking Time	2.45 [2.32–2.58] s	0.25 [0.22–0.28] s
SLA Compliance	80.8 [77–85]%	93.6 [91–96]%
Recovery Time	0.40 [0.36–0.44] s	0.16 [0.14–0.18] s
Modeled Throughput (ops/s)	408 [385–431]	4560 [4320–4800]

Mann–Whitney U test: $p < 0.001$ for all metrics (event-correlated vs. synchronous). Event-correlated agents showed improved resilience: 90% reduction in blocking time, 13 percentage points higher SLA compliance, and higher modeled operation throughput under $N = 10$ concurrent workflows. In this simulation, throughput denotes modeled operation completions per second under the configured event-processing workload rather than measured production deployment throughput. The event-correlated design improves this modeled throughput by overlapping I/O across concurrent workflows instead of blocking sequentially.

C. Discussion of Results

Under 20% failure injection, the Event-Related agent recovered in 0.16 s on average versus 0.40 s for the synchronous agent, which blocks for full retry latency. The 60% reduction in recovery time reflects the event model’s ability to handle retries asynchronously. The Correlation Context Store enables deterministic replay: on event arrival, the agent resumes from the exact state prior to intent publication.

VIII. DISCUSSION

A. Benefits

The architecture provides: horizontal scalability (agents are stateless between intents); failure domain isolation (service failures do not corrupt agent state); deterministic event reconciliation (CID matching); reduced cascading failures; and improved reliability under the modeled failure conditions.

B. Limitations

(1) **Increased architectural complexity** — Five components and an event bus add deployment and operational overhead compared with a purely synchronous tool-calling agent. (2) **Event infrastructure dependency** — The system requires reliable event infrastructure such as Kafka, RabbitMQ, or cloud-native messaging services, including monitoring for delayed, failed, or dead-lettered events. (3) **Persistent state overhead** — The Correlation Context Store must scale with concurrent intents, retained metadata, and context retention policies. Storage growth can be bounded using TTL-based cleanup, archival of completed contexts, and compaction of historical execution metadata. (4) **Fault tolerance requirements** — Production deployments require durable intent publication, idempotent event processing, retry isolation, and fail-secure behavior when the context store is unavailable. (5) **Simulation validity** — The evaluation uses a discrete-event model rather than live enterprise deployment. Real deployments may exhibit different latency distributions, event-bus congestion, downstream service saturation, schema evolution, retry storms, and multi-region consistency issues.

C. Comparison with Prior Work

Unlike MCP’s default synchronous tool model [1], our design externalizes intent and uses events for completion. Unlike pure event-driven orchestration [10]–[12], we integrate agent reasoning and resumption. Unlike SLA-aware schedulers [16] that optimize capacity, we focus on intent-to-event correlation and agent resumption. Our work is complementary: the architecture can sit atop existing orchestration and scheduling layers.

D. Future Work

Formal verification of the correlation model; adaptive SLA learning mechanisms; distributed consensus-based reconciliation for multi-region deployment; and integration with emerging MCP server standards.

IX. CONCLUSION

This paper presents an Event-Related MCP Agent architecture that bridges the gap between synchronous agent tool models and asynchronous enterprise systems. By introducing intent publication, correlation-aware reconciliation, and resumable reasoning, the framework enables scalable, reliable, and SLA-compliant agent execution in distributed environments. The proposed architecture demonstrates measurable improvements in resilience, throughput, and operational stability. This model offers a practical and extensible foundation for enterprise-oriented agent deployment in cloud-native ecosystems, while requiring further validation in production-scale environments.

REFERENCES

- [1] Anthropic, “Model Context Protocol,” 2024. [Online]. Available: <https://anthropic.com/news/model-context-protocol>
- [2] B. Michelson, “Event-Driven Architecture Overview,” O’Reilly, 2016.
- [3] S. Newman, *Building Microservices*, 2nd ed. O’Reilly, 2021.
- [4] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly, 2017.
- [5] OpenAI, “Model Context Protocol (MCP),” OpenAI Agents SDK Documentation, 2024.
- [6] G. Hohpe and B. Woolf, *Enterprise Integration Patterns*. Addison-Wesley, 2003.
- [7] B. H. Sigelman et al., “Dapper, a large-scale distributed systems tracing infrastructure,” Google, Tech. Rep., 2010.
- [8] OpenTelemetry, “Context Propagation,” OpenTelemetry Documentation. [Online]. Available: <https://opentelemetry.io/docs/concepts/context-propagation/>
- [9] W3C, “Trace Context,” W3C Recommendation, 2020.
- [10] Temporal Technologies, “Temporal Workflow Engine,” 2024. [Online]. Available: <https://temporal.io>
- [11] Netflix, “Conductor,” Netflix OSS. [Online]. Available: <https://conductor.netflix.com>
- [12] Argo Project, “Argo Workflows,” 2024. [Online]. Available: <https://argoproj.github.io/argo-workflows/>
- [13] V. Cardellini, M. Colajanni, and P. S. Yu, “Dynamic Load Balancing on Web-Server Systems,” *IEEE Internet Computing*, vol. 3, no. 3, pp. 28–39, 1999.
- [14] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-Scale Cluster Management at Google with Borg,” in *Proc. EuroSys*, 2015.
- [15] H. Zhang et al., “Sinan: ML-Based and QoS-Aware Resource Management for Cloud Microservices,” in *Proc. ASPLOS*, 2022.
- [16] D. V. Kalaga et al., “HAVEN: Holistic Load Balancing and Auto Scaling in the Cloud,” in *Proc. IEEE CLOUD*, 2015, pp. 218–225.
- [17] V. Methuku, N. Kapavarapu, K. Dharamshi, and V. Venkatesan, “Streaming to Predictive: Automating Cloud AI Workflows in Transaction-Intensive Digital Ecosystems,” in *Proc. 2025 International Conference on Computing Technologies and Data Communication (ICCTDC)*, Hassan, India, 2025, pp. 1–7, doi: 10.1109/ICCTDC64446.2025.11158751.